

4SO24EC027	Chitthara K S
4SO24EC024	Chaman R Shetty
4SO24EC026	Chirag M N
4SO24EC044	Hithashree

Report planning

Start date:10 March 2026

End date:2 May 2026

Chaman:

Chapter 2:Literature review

Chapter 4:Leaf cell design(Shift registers with testbench)

Work completed by 2nd May 2026

Chirag:

Research papers

Chapter 1:Introduction

Work completed by 2nd May 2026

Chitthara:

Research papers

Chapter 3:Proposed system design(architectutral diagram)

Chapter 4:Leaf cell design(multipliers with testbench)

Work completed by 2nd May 2026

Hitha:

Chapter 2:Proposed system design(Block diagram)

Chapter 4:Leaf cell design(with testbench)

Work completed by 2nd May 2026

Adaptive FIR filter
using LMS algorithm

INTRODUCTION

Digital signal processing is widely used in applications such as communication systems, audio processing, and environmental monitoring. However, real-world signals are often affected by noise, which reduces system accuracy and performance. To address this issue, filtering techniques are used to extract useful information from noisy signals.

Problem Statement:

Real-time biomedical and environmental monitoring systems require efficient digital signal processing. Software-based FIR filtering can introduce latency and increased power consumption in wearable and IoT devices. Design and implement an FPGA-based hardware system capable of performing real-time digital filtering for streaming input data.

Objectives:

The design focuses on developing an adaptive FIR filter architecture based on the LMS algorithm, optimized for efficient FPGA implementation. The filter employs a shift register-based delay line to store input samples, reducing resource utilization and improving hardware efficiency.

Comparative Analysis Table

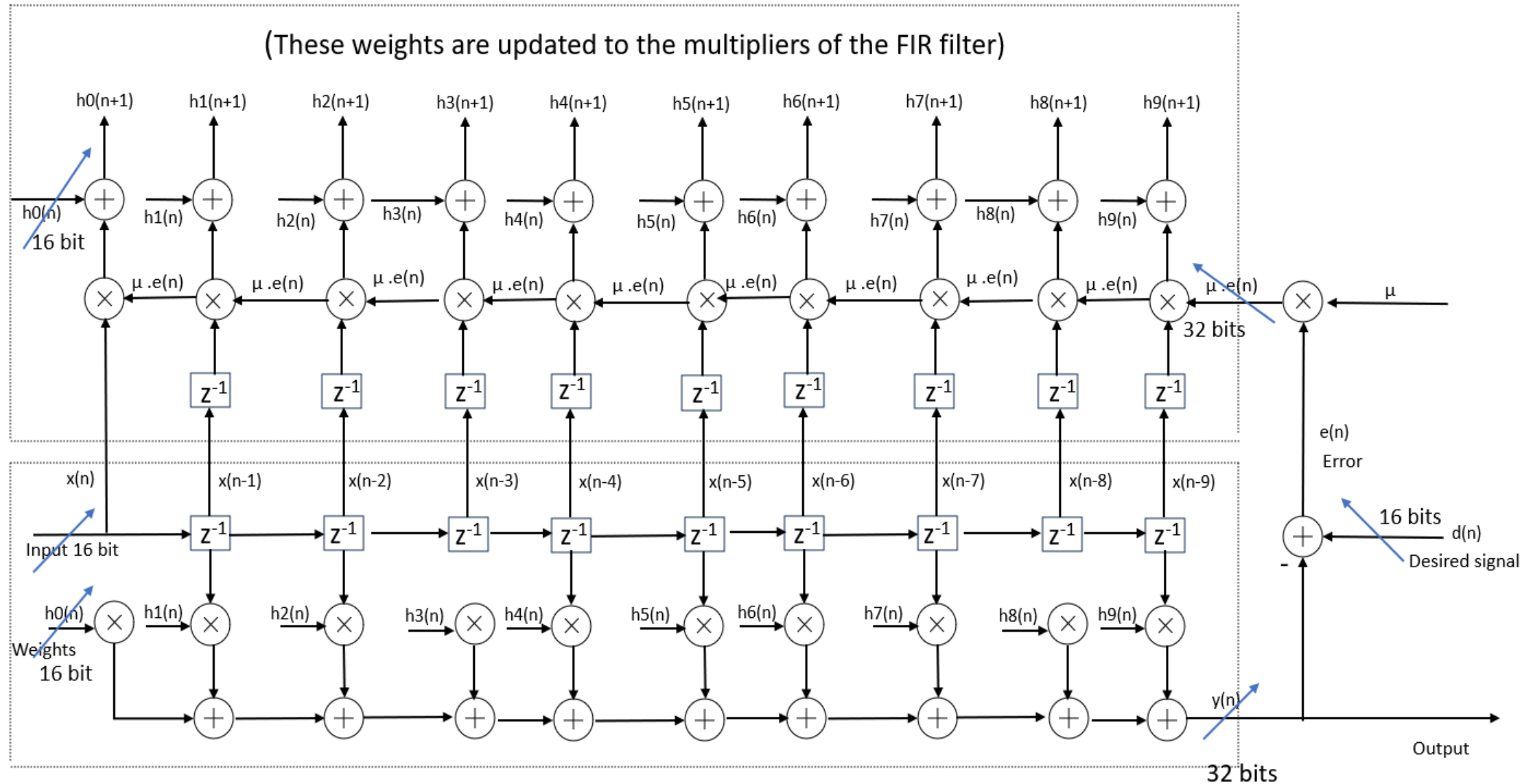
Paper	Architecture	Throughput (Approx.)	Area (Approx.)	Limitation
1	FPGA LMS (BCD Multiplier)	~1282 MHz (max clock speed)	151 logic elements 12 registers	Only works for small filter sizes No power analysis done
2	LMS MAC-based	~600 MHz	~70 logic elements (~46% of Paper 1)	Precision can be lower Performance depends on convergence
3	DA-based LMS	~900 MHz (~18% faster than Paper 2)	~8 logic elements (~5% of Paper 1)	Higher latency Uses more memory
4	DA + LMS Hybrid	~700 MHz	~20 logic elements (~13% of Paper 1)	Design is complex May have latency issues

This comparison helps identify the trade-off between speed, hardware utilization, and design complexity in different LMS architectures.

- 1.Highest Speed: FPGA LMS (BCD Multiplier)
- 2. Least Area: DA-based LMS
- 3. Best Trade-off: DA + LMS Hybrid

Architectural Diagram

LMS Algorithm



FIR Filter

Software used:

- MATLAB/Simulink:

Used for testing the ANC filter

Helps to check noise cancellation

- Xilinx Vivado:

Used for Verilog coding

Used for simulation

- Power Point:

Used for making diagrams and presentation

System Architecture

1. Pipelined Multiplier

Used to multiply: Input samples $\times$ filter coefficients

Error signal $\times$ input signal during LMS coefficient update

In FIR filtering:

$y(n) = \sum_{k=0}^{N-1} w_k(n) \times (n-k)$ each tap requires multiplication.

Why pipelined multiplier is used:

1. Increases processing speed
2. Supports real-time signal processing
3. Reduces critical path delay
4. Efficiently utilizes FPGA DSP slices
5. Allows higher clock frequency

2) Carry Look Ahead Adder (CLA)

Used to: Add outputs of all multipliers
Compute filter output
Calculate error signal

Error equation:

$$e(n) = d(n) - y(n)$$

Why CLA is used:

Faster than ripple carry adder
Reduces carry propagation delay
Improves overall FPGA speed
Suitable for high-speed DSP systems

Because FIR filters require continuous additions, fast adders are important.

3) Shift Registers

Used to:

Store previous input samples

Create delayed versions of the input signal

Delay operation:

$x(n)$, $x(n-1)$, $x(n-2)$

Why shift registers are used:

FIR filters need past input samples

Each tap uses a delayed sample

Enables sequential data flow

FPGA SRL resources implement delays efficiently

Without shift registers, the filter cannot access previous samples

4) Error Calculation Block

Used to compute difference between:

Desired signal

Actual filter output

Error equation:

$$e(n)=d(n)-y(n)$$

Why it is used:

Measures filter performance

Indicates how close output is to desired signal

Drives coefficient adaptation

Essential for LMS learning process

The adaptive filter improves itself using this error value.

5) LMS Update Block

Used to continuously update filter coefficients (weights).

LMS update equation:

$$h(n+1) = h(n) + \mu x(n) e(n)$$

Why LMS block is used:

Makes filter adaptive

Minimizes error over time

Automatically adjusts coefficients

Helps track changing signals/noise

Core learning unit of adaptive FIR filter

This block is what differentiates an adaptive FIR filter from a normal FIR filter

Github repository

`https://github.com/chittharaks/Adaptive-FIR-filter-using-LMS-algorithm1.git`

 **Adaptive-FIR-filter-using-LMS-algorithm1** Public

 Pin  Watch 0  Fork 0  Star 0

main 1 Branch 0 Tags Add file Code

 **chittharaks** Delete doc/r f3f4c52 · 7 hours ago 31 Commits

 Testbench	Delete Testbench/Adder(TB)	8 hours ago
 doc	Delete doc/r	7 hours ago
 src	Create Adder	8 hours ago
 LICENSE	Initial commit	14 hours ago
 README.md	Initial commit	14 hours ago

 **README**  Apache-2.0 license 

Adaptive-FIR-filter-using-LMS-algorithm1

About

No description, website, or topics provided.

-  Readme
-  Apache-2.0 license
-  Activity
-  0 stars
-  0 watching
-  0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)